

***Pathfinder* and Other Tools for Analyzing Signal Transduction Networks**

LARRY LOK

The Molecular Sciences Institute, Berkeley, California 94704, USA

ABSTRACT: Many electronic design tools are, at their core, tools for analyzing interacting networks of components in which there is a definite causal sense of direction in each interaction: outputs affect inputs, but not vice versa. Simplifying or restricting a network of protein interactions so that this kind of directedness holds makes the same category of software tools applicable. In signal transduction cascades the direction is usually given by “kinase phosphorylates substrate.” This paper examines a few kinds of directed network analysis tools in the context of signal transduction.

KEYWORDS: *Pathfinder*; signal transduction network; qualitative differential equation schemes

INTRODUCTION

The programs used to analyze and validate digital electronic designs, or delay logic networks, generally fall into two classes, static network analysis tools and simulations. By static analysis, the timing properties of circuits without feedback can be validated without the need for simulation. This achieves a considerable savings in computational speed. Because many digital circuits can be broken down into large, feedback-free chunks, one gets a big part of circuit validation at a computational discount over an approach that uses only simulation.

The same classes of programs are coming to bear on complex networks of interacting proteins in cells. The possibility of simulating, for example, protein interactions, binding and unbinding, and phosphorylation at various levels of abstraction has become generally known in the biological community. The advantages and disadvantages of static network tools are perhaps less well known. I have written a simple program called *Pathfinder* to do a static analysis of the connectivity of feedback-free transduction networks. One purpose of the program and of this article is to highlight the static network software category, as well as some other “neglected” kinds of software for analyzing complex network behavior.

Address for correspondence: Larry Lok, The Molecular Sciences Institute, 2168 Shattuck Avenue, Berkeley, CA 94704. Voice: 510-981-8740; fax: 510-647-0699.
lok@molsci.org

Ann. N.Y. Acad. Sci. 971: 589–594 (2002). © 2002 New York Academy of Sciences.

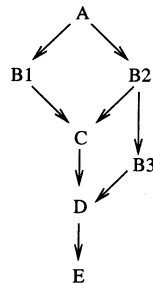


FIGURE 1. A simple network.

PATHFINDER FUNCTIONALITY

Pathfinder was created to help locate in a signal transduction network a set of proteins, which, when knocked out, prevent a given protein from being activated. This can be tricky to do by hand, even for relatively simple networks. There is one network where the solution is easy: a simple chain of proteins, each one activating the next. Knocking out any element of the chain will deactivate all downstream proteins. To find small knockout sets in a more complex network, *Pathfinder*, in effect, finds all the paths from a user-specified set of “activated” proteins in the network to another user-specified set of “target” proteins. This done, what remains is to select a knockout set that will disrupt each of these paths. This can sometimes be done by hand; a simple “greedy” type algorithm is effective in general.

FIGURE 1 shows a simple network and its *Pathfinder* specification is as follows:

```

B1 A
B2 A
C B1
C B2
B3 B2
D C
D B3
E D
  
```

If we designate A as the source protein and E as the target, then *Pathfinder* finds the obvious paths through the network:

```

E D C B1 A
E D C B2 A
E D B3 B2 A
  
```

and it is evident that D is the only single knockout that will deactivate E.

Pathfinder can show us the result of knockouts without reentering the network. For example, if B3 is knocked out, then only two paths are found through the network:

```

E D C B1 A
E D C B2 A.
  
```

Finally, *Pathfinder* can treat a set of “ancestor” proteins’ simultaneous activation as necessary for the activation of a single “child” protein. To illustrate, we modify

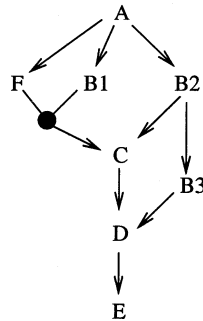


FIGURE 2. Simultaneous activation.

our sample network to convey that C can be activated by B1 only if F is also active (FIG. 2).

Now the specification looks like this:

```

B1 A
B2 A
F A
C B1 F
C B2
B3 B2
D C
D B3
E D

```

and the *Pathfinder* output looks like this:

```

E D C B1 A
E D C F A
E D C B2 A
E D B3 B2 A

```

The first two lines, a “multipath,” show the dependency of C on both B1 and F, and knocking out F is enough to make the multipath go away:

```

E D C B2 A
E D B3 B2 A

```

LIMITATIONS

Two main practical limitations of *Pathfinder* are that it does not understand inhibitory links in a network and that it is limited to networks that are “directed acyclic.” That is, one cannot follow arrows around a closed loop.

Inhibitory links imply quantitation of activation and signals. It seems reasonable to expect that some useful and systematic way of propagating notions like “degrees of activation,” “strongly activating,” and “weakly inhibiting” through the network might be achieved in a more complex program of the same kind as *Pathfinder*.

However, limitation of *Pathfinder* to directed acyclic graphs is deeper. The reason is that a closed “activation loop” makes sense only if interpreted as involving a delay, and delays are not within *Pathfinder*’s simple scope. Treating such delays naturally leads to simulation.

CAUSAL NETWORKS

A statistical approach that falls under the heading of static network analysis is the “causal network” theory of Pearl.¹ The practice of causal network theory is far beyond the scope of this article. The general idea is to represent hypothetical causal relations between random variables by a directed graph such as FIGURE 1. We then would “read” that graph as saying “activation of A causes activation of B1,” “activation of B1 causes activation of C,” and so on. The connectivity of the graph then is used systematically to construct statistical procedures that (1) test experimental data for consistency with the causal hypothesis given by the graph; and (2) search systematically for dependencies (typically correlations) among experimental data that cannot be explained by the causal hypothesis given by the graph.

Just as in static timing analysis and in *Pathfinder*, causal graphs generally are limited to directed acyclic graphs. In this case, a closed loop would translate into a closed causal loop, and implicitly, a self-causing event. The only way to make sense of such a loop is by “reading in” a delay, so that the event that is the effect is not the same event as the cause, but rather a later one. This makes the separation of events in time essential, and the causal network scheme starts to expand into simulation-like complexity.²

Because of their statistical character, causal networks generally are not used for the same engineering goal as, for example, static timing analysis: the predictive validation of a proposed, but as yet nonexistent, design. Rather, their application is usually to the scientific goal of refining hypotheses and testing them against experimental data.

SIMULATION

Simulation, in which the program explicitly follows events through time, is in general much more computationally intensive than static analysis. As computer programs, simulations are generally more complex and harder to use. Most significantly, a vast new category of dynamical data becomes necessary to support simulation: the dynamical description of exactly how events follow one another. For biological systems, this dynamical data amounts to reaction rates for all the reactions in the system. Reaction rate data are difficult to come by in the literature and difficult to acquire in the laboratory. For this reason, simulation methodologies that are not completely stymied by a partial lack of dynamical data (reaction rates) are of definite interest. Failing the ability to go on despite a lack of rate data, we would at least like for the tool to help us in the process of determining the rates. Here, we briefly look at three simulation approaches to signal transduction networks with regard to data sufficiency, results, and computational intensity:

1. Traditional ordinary differential equation (ODE) solvers
2. Stochastic reaction event simulation
3. Qualitative differential equations schemes.

In the first and best-known approach, signal transduction networks are simulated with traditional ODE solvers by applying the solver to the mass-action equation. The reaction rates of all the reactions in the coupled system are essential data, along with the initial concentrations of all the proteins in the network. The result is (generally) an equally spaced time series of concentrations predicting the evolution of the system. Of the three methodologies listed, this one is the oldest and generally the fastest.

Stochastic reaction event simulation³ uses basically the same data as traditional ODE solvers, but by dint of higher computational intensity obtain a more detailed simulation. Once again, reaction rates of all reactions are essential data, along with the initial number of molecules of each protein in the network, rather than protein concentrations. The simulation then generates a sequence of reaction events, each of which adjusts the molecular populations by integer amounts. Some provision for dumping populations at regular intervals is generally made, yielding output of the same kind as the ODE approach. Tellingly, some of these simulators make no provision for examining the raw event stream into which so much computational energy has been put. Touching each individual reaction event is the source of the computational intensity of this approach. The predicted sequence of reaction events is not deterministic; rather, it is a stochastic process whose probable evolution is determined by reaction rate data. One expects in general that averaging many stochastic reaction event simulations should produce a summary similar to the output of an ODE solver.

Stochastic reaction event simulation is desirable in situations which very few molecules of network proteins can affect the overall course of simulation, so that keeping a precise count is important. Another possible benefit of the stochastic/statistical approach is that there is at least a signpost pointing toward a way to assist in determination of rate data: statistical inference. Such a simulator is, abstractly, a statistical model whose parameters include the reaction rates. As such, one might hope to be able to confront it with experimental data similar to simulation output and from that data to infer the parameters, the reaction rate data, at least in simple cases. Unfortunately, using a statistical model for simulation and using it for inference require different software constructs.

Stochastic reaction event simulation is currently the simulation workhorse of The Molecular Sciences Institute's efforts to explore and elucidate the alpha-factor pathway in yeast.

The practice of "qualitative differential equation" systems⁴ is again beyond this article's scope, but they are essentially expert systems to do a sophisticated form of an activity taught in every calculus class: using information about the signs of quantities and their derivatives, together with some basic theorems of calculus, to infer the overall shape of the graph of the quantities against time. For example, by observing that $f(-1) = f(1) = 0$ and $f(x) > 0$ for $-1 < x < 1$, we can conclude that $f'(c) = 0$ for some c in the interval $(-1, 1)$. Given the data, such reasoning leads to a list of qualitative predictions of system behavior, or "scenarios," in which each scenario comes from one possible set of assumptions filling in missing data. For instance, if there were another function in the simulation, $g(x)$, satisfying the same constraint as f

above, we would similarly have $g'(d) = 0$ for some d in $(-1, 1)$. We might get two scenarios depending on whether $c < d$ or $c > d$.

Qualitative differential equation schemes are designed to deal with incomplete data, but they are computationally intensive even for relatively small systems of ODEs. Also, responding to missing data by evenhandedly exploring all possible worlds does not seem as though it would be particularly useful for a large network, assuming that it could be carried out in practice.

REFERENCES

1. PEARL, J. 2000. *Causality: Models, Reasoning, and Inference*. Cambridge University Press. Cambridge, United Kingdom.
2. SHIPLEY, S. 2000. *Cause and Correlation in Biology: A User's Guide to Path Analysis, Structural Equations, and Causal Inference*. Cambridge University Press. Cambridge, United Kingdom.
3. GILLESPIE, D.T. 1976. A general method for numerically simulating the stochastic time evolution of coupled chemical reactions. *J. Comput. Physics* **22**: 403–434.
4. KUIPERS, B. 1994. *Qualitative reasoning: modeling and simulation with incomplete knowledge*. MIT Press. Cambridge, MA.